Deep Reinforcement Learning: Policy Gradient and Its Applications

Yuanchu Liang   Jingyang You

The Australian National University

yuanchu.liang@anu.edu.au   jingyang.you@anu.edu.au

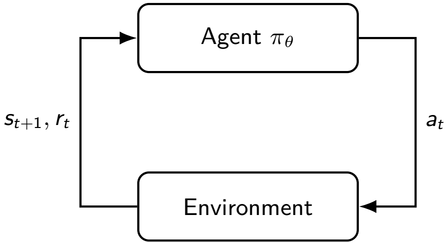
Why Reinforcement Learning?

Core setting: an agent repeatedly acts, observes consequences, and improves its behaviour from scalar feedback.

- **Robotics:** locomotion, manipulation, navigation.
- **Games:** Go, Atari, Mahjong, real-time strategy.
- **Sequential decisions:** recommendation, dialogue, resource allocation.

Distinctive difficulty:

- actions affect future data distribution;
- rewards may be delayed and sparse;
- exploration and exploitation are coupled.



Learning problem

Choose a policy that maximises long-term reward, not just immediate prediction accuracy.

Markov Decision Processes: Formal Model

A discounted Markov decision process is

$$\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, r, \rho_0, \gamma), \qquad \gamma \in [0, 1).$$

- $s_0 \sim \rho_0$, $a_t \sim \pi_\theta(\cdot \mid s_t)$, $s_{t+1} \sim P(\cdot \mid s_t, a_t)$.
- $r(s_t, a_t)$ is the one-step reward; stochastic rewards can be included by taking conditional expectation.
- The Markov property: future dynamics depend on history only through (s_t, a_t).

For a trajectory $\tau = (s_0, a_0, s_1, a_1, \ldots)$,

$$p_\theta(\tau) = \rho_0(s_0) \prod_{t \geq 0} \pi_\theta(a_t \mid s_t) P(s_{t+1} \mid s_t, a_t).$$

Control objective

$$J(\theta) = \mathbb{E}_{\tau \sim p_\theta} \left[\sum_{t=0}^{\infty} \gamma^t r(s_t, a_t) \right].$$

Value Functions and Bellman Equations

For a fixed policy π, define

$$V^\pi(s) = \mathbb{E}_\pi\left[\sum_{k=0}^{\infty} \gamma^k r(s_{t+k}, a_{t+k}) \mid s_t = s\right],$$

$$Q^\pi(s,a) = \mathbb{E}_\pi\left[\sum_{k=0}^{\infty} \gamma^k r(s_{t+k}, a_{t+k}) \mid s_t = s, a_t = a\right].$$

Policy evaluation:

$$V^\pi(s) = \mathbb{E}_{a\sim\pi}\left[r(s,a) + \gamma \mathbb{E}_{s'\sim P} V^\pi(s')\right].$$

$$Q^\pi(s,a) = r(s,a) + \gamma \mathbb{E}_{s'\sim P, a'\sim\pi} Q^\pi(s',a').$$

Optimality:

$$V^*(s) = \max_a \left\{r(s,a) + \gamma \mathbb{E}_{s'\sim P} V^*(s')\right\}.$$

$$Q^*(s,a) = r(s,a) + \gamma \mathbb{E}_{s'\sim P} \max_{a'} Q^*(s',a').$$

Intuition

Bellman equations express a global return as immediate reward plus the value of the next decision point.

Policy Gradient

Policy gradient methods directly optimise a differentiable stochastic policy $\pi_\theta(a \mid s)$. Write the objective as an integral over trajectories,

$$J(\theta) = \int p_\theta(\tau) R(\tau)\, \mathrm{d}\tau, \qquad R(\tau) = \sum_{t=0}^{\infty} \gamma^t r(s_t, a_t). \qquad p_\theta(\tau) = \rho_0(s_0) \prod_{t \geq 0} \pi_\theta(a_t \mid s_t) P(s_{t+1} \mid s_t, a_t).$$

Gradient-based Optimisation. It is natural to consider the gradient of the above formula

$$\nabla_\theta J(\theta) = \int \nabla_\theta p_\theta(\tau)\, R(\tau)\, \mathrm{d}\tau$$

However, the integrand cannot be computed/approximated directly. The log-trick makes it more useful

$$\nabla_\theta p_\theta(\tau) = p_\theta(\tau)\, \nabla_\theta \log p_\theta(\tau). \qquad \nabla_\theta J(\theta) = \int p_\theta(\tau)\, \nabla_\theta \log p_\theta(\tau)\, R(\tau)\, \mathrm{d}\tau = \mathbb{E}_{\tau \sim p_\theta} \left[\nabla_\theta \log p_\theta(\tau)\, R(\tau) \right].$$

The gradient has become an expectation that can be Monte-Carlo approximated. Let's further simplify this term.

Policy Gradient

Now we are at

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim p_\theta} \left[\nabla_\theta \log p_\theta(\tau) \, R(\tau) \right].$$

Note that

$$\log p_\theta(\tau) = \log \rho_0(s_0) + \sum_{t \geq 0} \Big[\log \pi_\theta(a_t \mid s_t) + \log P(s_{t+1} \mid s_t, a_t) \Big].$$

Only the policy terms depend on θ; the gradients of $\log \rho_0$ and $\log P$ vanish:

$$\nabla_\theta \log p_\theta(\tau) = \sum_{t \geq 0} \nabla_\theta \log \pi_\theta(a_t \mid s_t).$$

Further simplifying on the indices

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim p_\theta} \left[\sum_{t \geq 0} \gamma^t \, \nabla_\theta \log \pi_\theta(a_t \mid s_t) \, G_t \right], \qquad G_t = \sum_{k \geq t} \gamma^{k-t} r_k.$$

Policy Gradient Theorem

We are at

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim p_\theta} \left[\sum_{t \geq 0} \gamma^t \, \nabla_\theta \log \pi_\theta(a_t \mid s_t) \, G_t \right], \qquad G_t = \sum_{k \geq t} \gamma^{k-t} r_k .$$

As $\mathbb{E}\left[G_t \mid s_t, a_t\right] = Q^{\pi_\theta}(s_t, a_t)$

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim p_\theta} \left[\sum_{t \geq 0} \gamma^t \, \nabla_\theta \log \pi_\theta(a_t \mid s_t) \, Q^{\pi_\theta}(s_t, a_t) \right] .$$

Policy Gradient Theorem

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim p_\theta} \left[\sum_{t \geq 0} \gamma^t \, \nabla_\theta \log \pi_\theta(a_t \mid s_t) \, Q^{\pi_\theta}(s_t, a_t) \right]$$

REINFORCE, Baselines, and Variance Reduction

REINFORCE proposes a naive Monte-Carlo estimate of $\nabla_\theta J(\theta)$, where

$$\widehat{g}_{\mathrm{RF}} = \frac{1}{N} \sum_{i=1}^{N} \sum_{t \geq 0} \gamma^t \, \nabla_\theta \log \pi_\theta(a_t^{(i)} \mid s_t^{(i)}) \, \widehat{Q}_t^{(i)}, \text{ where } \widehat{Q}_t = \sum_{k=t}^{T-1} \gamma^{k-t} r(s_k, a_k).$$

However, the variance of such estimation is very high.

Policy Gradient with Baselines

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim p_\theta} \left[\sum_{t \geq 0} \gamma^t \, \nabla_\theta \log \pi_\theta(a_t \mid s_t) \, \left(Q^{\pi_\theta}(s_t, a_t) - b(s_t)\right) \right]$$

Baseline A state-dependent baseline does not change the expected gradient:

$$\mathbb{E}_{a_t \sim \pi_\theta(\cdot \mid s_t)} \left[\nabla_\theta \log \pi_\theta(a_t \mid s_t) b(s_t) \right] = b(s_t) \nabla_\theta \int_a \pi_\theta(a \mid s_t) \, da = 0.$$

Advantage $b(s_t) = V^{\pi_\theta}(s_t)$ is a common choice for a reduced variance. $A^{\pi_\theta}(s_t, a_t) := Q^{\pi_\theta}(s_t, a_t) - V^{\pi_\theta}(s_t)$ is named Advantage.

Advantage Estimation in Practice

Now the problem becomes how to estimate $A_t^{\pi\theta} = Q^{\pi\theta}(s_t, a_t) - V^{\pi\theta}(s_t)$.

Temporal Difference (TD). We note that

$$\begin{aligned} A_t^{\pi\theta} &= r(s_t, a_t) + \gamma \mathbb{E}_{s' \sim P}[V^{\pi\theta}(s')] - V^{\pi\theta}(s_t) \\ &\approx r(s_t, a_t) + \gamma V^{\pi\theta}(s_{t+1}) - V^{\pi\theta}(s_t) := \widehat{A}_t^{(1)} \\ &\approx r(s_t, a_t) + \gamma\, r(s_{t+1}, a_{t+1}) + \gamma^2\, r(s_{t+2}, a_{t+2}) + \cdots + \gamma^n V^{\pi\theta}(s_{t+n}) - V^{\pi\theta}(s_t) := \widehat{A}_t^{(n)} \end{aligned}$$

We name $\widehat{A}_t^{(1)}$ a **one-step TD** estimate, $\widehat{A}_t^{(n)}$ a **n-step TD** estimate of $A_t^{\pi\theta}$.

Generalized Advantage Estimation (GAE). Weighted average of many $\widehat{A}_t^{(n)}$ at different n.

$$\widehat{A}_t^{\mathrm{GAE}} = (1 - \lambda) \sum_{n \geq 1} \lambda^{n-1} \widehat{A}_t^{(n)}$$

Both methods require $V^{\pi\theta}(\cdot)$, which is not directly accessible. In practice this is usually approximated using a neural network $V_\phi(\cdot)$.

Issues with Policy Updates

Suppose one has obtained $\nabla_\theta J(\theta)$ and would like to use it for policy updates, i.e.

$$\theta_{\text{new}} \leftarrow \theta_{\text{old}} + \alpha \cdot \nabla_\theta J(\theta)|_{\theta=\theta_{\text{old}}}$$

Such naive gradient update is only safe to perform once. **Why?**

Because the data (trajectories) are dependent on θ. An updated θ_{new} needs to be updated from gradients computed from new trajectories which are collected following $\pi_{\theta_{\text{new}}}$.

Is there a more controlled way to perform gradient update multiple times using the same batch of data?

Proximal policy optimisation (**PPO**) considers optimising instead a surrogate objective function that allows safer multiple gradient updates.

Policy Ratios and Surrogate Objectives

In PPO (Schulman et al. 2017), for a fixed s_t, the previous objective wants to estimate the expected advantage,

$$\mathbb{E}_{a_t \sim \pi_\theta(\cdot|s_t)}[\widehat{A}_t] = \int \pi_\theta(a_t \mid s_t)\widehat{A}_t(s_t, a_t)\mathrm{d}a_t$$

using the new policy π_θ but the data we collected so far is from π_{old}. Hence we multiply and divide by π_{old} to get,

$$\mathbb{E}_{a_t \sim \pi_\theta(\cdot|s_t)}[\widehat{A}_t] = \int \pi_{\mathrm{old}}(a_t \mid s_t)\frac{\pi_\theta(a_t \mid s_t)}{\pi_{\mathrm{old}}(a_t \mid s_t)}\widehat{A}_t(s_t, a_t)\mathrm{d}a_t = \mathbb{E}_{a_t \sim \pi_{\mathrm{old}}}\left[\frac{\pi_\theta(a_t \mid s_t)}{\pi_{\mathrm{old}}(a_t \mid s_t)}\widehat{A}_t(s_t, a_t)\right]$$

We define $r_t(\theta) := \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\mathrm{old}}}(a_t|s_t)}$. In reality the magnitude of r_t needs to be controlled to ensure each gradient do not take too big leap, hence, we also apply clipping to the ratio,

$$\mathrm{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\widehat{A}_t \tag{1}$$

PPO Objective and Training Recipe

The clipped surrogate objective is

$$L^{\mathrm{CLIP}}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta)\widehat{A}_t, \mathrm{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\widehat{A}_t \right) \right].$$

- If $\widehat{A}_t > 0$, PPO prevents excessively increasing the action probability.
- If $\widehat{A}_t < 0$, PPO prevents excessively decreasing the action probability.
- The minimum makes the objective pessimistic relative to the unclipped surrogate.

A common implementation optimises

PPO Objective

$$\max_{\theta,\phi} \quad \underbrace{L^{\mathrm{CLIP}}(\theta)}_{\text{policy gradient}} - c_v \underbrace{\mathbb{E}_t \left[\left(V_\phi(s_t) - \widehat{G}_t \right)^2 \right]}_{\text{value regression}} + c_H \underbrace{\mathbb{E}_t \left[\mathcal{H}(\pi_\theta(\cdot \mid s_t)) \right]}_{\text{maximum entropy}}.$$

Typical Work Flow. Collect trajectories on-policy, compute $\widehat{A}_t$, run several minibatch epochs, then discard the data.

Group Relative Policy Optimisation (GRPO)

The capability of automatically explore and exploit in RL finds great success in fine-tuning pre-trained models.
In Shao et al. 2024, given a pre-trained LLM and a prompt x, GRPO generates k responses from the model, $y_1, \ldots, y_k$ and assign a reward $r_1, \ldots, r_k$ for each response. They then define the group relative advantage,

$$\widehat{A}_i = \frac{r_i - \mathrm{mean}(r_1, \ldots, r_k)}{\mathrm{std}(r_1, \ldots, r_k) + \epsilon}$$

And optimise the following objective,

$$\mathcal{L}_{\mathrm{GRPO}}(\theta) = \mathbb{E}_{x, \{y_i\}_{i=1}^{G}} \left[\frac{1}{G} \sum_{i=1}^{G} \mathsf{min} \left(\rho_i(\theta) \hat{A}_i, \, \mathsf{clip} \left(\rho_i(\theta), 1 - \varepsilon, 1 + \varepsilon \right) \hat{A}_i \right) - \beta D_{\mathrm{KL}} \left(\pi_\theta(\cdot \mid x) \, \| \, \pi_{\mathrm{ref}}(\cdot \mid x) \right) \right]$$

where ρ_i is the policy ratio for each generated response.

Diffusion Policy Policy Optimisation (DPPO)

In Ren et al. 2025, they consider the Markov chain generated by a pre-trained diffusion policy as part of the MDP environment and apply PPO directly to fine tune the parameters of a diffusion policy.

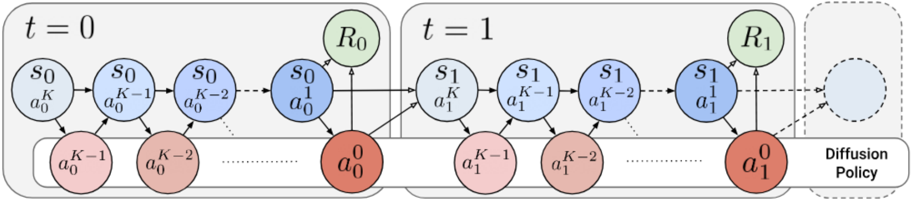


Figure: DPPO flow diagram. Subscripts denote environment steps and superscripts denote diffusion sampling steps. Each block is a diffusion generation process. When an action is generated, it is executed in the environment to transit the state to the next state.